The dependency graph as a propagation engine

This document details point 4.4 of `comparison_team.md`: today the dependency graph is a picture, while the teammates propose to make it the **engine** that computes losses. It explains what that means, measures on the real example graph what the picture already reveals, works through one shock end to end, and describes how the engine could be built in steps. A learn section at the end explains the concepts used.

The data is the example graph `docs/graph-example.json`: 100 nodes and 134 edges at block 26,162,421 (2026-10-10). It holds 19 actors (the demo borrower, Sentora RLUSD Main and its largest depositor, and the main lenders of the borrowed markets) and 16 Blue markets. The losses of the worked example come from the tool's `market_risk.value()` on the vault's borrowers recorded at block 26,162,932, a few hundred blocks later.

Contents

- 1. The question
- 2. What the tool does today
- 3. What the teammates propose
- 4. Transmission functions for this graph
- 5. Limiting the spread: DebtRank
- 6. Worked example on the real graph
- 7. What could be implemented
- 8. Limits
- Learn: what you need to understand

1. The question

When one thing fails (a price feed, a collateral token, a Pendle pool, a market), **who loses, and how much?** The question must be answerable for any node of the graph, and for a protocol the tool has never seen, without writing new loss code for it.

2. What the tool does today

- **`graph.py`** builds the graph from live data: typed nodes (Actor, Pool, Asset, Oracle, Admin, External) and edges carrying the amount that flows along them (a vault's supply in a market, a borrower's debt). It draws it in `graph.html`.
- **The losses are computed elsewhere,** by code written for one situation:
  - `propagate.py` knows the five layers of one Pendle PT borrow;
  - `market_risk.py` knows how a price fall becomes bad debt in one market;
  - the alert rules know vaults and borrows.
- **None of them reads the graph.** A shock on a node the code was not written for (an issuer's custodian, a shared feed) produces no number.

This is gap R1 of `ANALYSIS_MONITORING.md`. Its visible symptom is gap R3: the graph shows that one `RLUSD / USD` feed sits under all of Sentora's markets, but nothing measures what that means.

3. What the teammates propose

Team 2 (and Team 1 in other words: look-through exposure, blast radius):

1. **Every edge type gets a transmission function**: how a shock on the node at the end of the edge becomes a loss on the node at its start.
2. **Shocks are failure modes, not single numbers**: a price falls by x, a token depegs by x, a feed freezes, misreports by ±x or goes stale, a contract suffers a total loss. An oracle does not "lose 20 %": it reads too high, too low, or stops.
3. **A generic engine spreads the shock** through the graph with **DebtRank** (Battiston et al., 2012). Each node passes a shock on once, which handles cycles, small paths are pruned, and terminal nodes such as USDC and Ethereum stop the spread.
4. **Adding a protocol** then means producing nodes and edges of existing types; the engine and the decision layer do not change.

4. Transmission functions for this graph

The edges `graph.py` already produces, with the function each would carry. Most of the maths exists in the code; what is missing is attaching it to the edges.

Edge (start → end)	**A shock on the end node…**	**…becomes on the start node**	**Exists today in**
actor (vault or lender) → market, *supplies*	bad debt BD in the market	loss = share of the market's supply × BD	`market_risk` (vault side)
actor (borrower) → market, *borrows from*	a fall of the market's oracle price	loss of the position: none above the liquidation distance, then liquidation (`propagate.loss`)	`propagate`, `borrower`
actor (depositor) → vault, *deposits in*	vault loss L	loss = depositor's share of the vault × L	`market_risk.assess_vault_bad_debt`
market → collateral asset, *takes as collateral*	collateral price falls by x	the oracle price falls by x, if the oracle follows that asset; then bad debt from the market's borrowers	`market_risk.value`

Edge (start → end)	A shock on the end node…	…becomes on the start node	Exists today in
market → oracle, *priced by*	oracle misreports by −x / +x / freezes	−x: like a collateral fall (liquidations, no real loss of value); +x: borrowers can borrow too much, a loss only if exploited; freeze: no liquidations, bad debt builds silently	partly (`propagate`, oracle event)
oracle → feed, *reads*	feed misreports by x	a base feed: oracle price × (1 + x); a quote feed: oracle price ÷ (1 + x)	not yet
oracle → vault share, *converts through*	share's exchange rate falls by x	oracle price × (1 − x)	not yet
collateral PT → Pendle pool → yield token	yield token depegs by x	PT value falls by about x (more during a run on the pool)	`propagate`
yield token → off-chain dependency	custodian or exchange failure	a scenario from `registry.json` (depeg x %, loss of backing)	`registry.json`
admin → vault / contract	upgrade or malicious change	up to total loss, capped when the timelock is longer than the depositor's reaction time	alerts only

The functions are **not linear**: a borrower loses nothing until the liquidation distance and almost everything just after it, and a market makes no bad debt until the fall passes its insolvency point. That is why the engine must call the existing functions rather than multiply percentages along the path.

5. Limiting the spread: DebtRank

A shock on a node reaches everything that depends on it, directly or not. Without rules, every shock reaches every node, because almost every market touches USDC and Ethereum. DebtRank, a method built for contagion between banks, gives three rules:

1. **Relative distress.** Each node has a distress h between 0 (unaffected) and 1 (lost everything). A node's distress is the loss it suffers divided by its own size.
2. **Each node passes its shock on once.** When a node becomes distressed, it transmits to the nodes that depend on it, then becomes inactive. A cycle (a vault share used as collateral in a

market the same vault supplies) cannot amplify a shock forever.
3. **Pruning.** A path that carries less than a threshold ε of a node's value is dropped.

The update, in the paper's form, with W_ij the weight of i's impact on j (j's exposure to i divided by j's own value):

$$h_j(t) = \min\left(1,\ h_j(t-1) + \sum_{i \text{ distressed at } t-1} W_{ij}\, h_i(t-1)\right)$$

Here, W_ij would be replaced by the transmission function of the edge, because the losses are non-linear.

A small example. A feed fails. Market A loses 10 % of its supply to bad debt and market B 5 %. A vault holds 60 % of its assets in A and 40 % in B. The vault's distress is 0.6 × 10 % + 0.4 × 5 % = 8 %. A depositor holding 30 % of the vault loses 8 % of its deposit, and the vault, having passed its distress on once, does not pass it again.

6. Worked example on the real graph

Step 1, without an engine: shared-node exposure

For every node, sum the amounts of the actors' positions in markets that depend on it. This needs only the existing `graph.json` and a reachability search. The nodes carrying the most:

Node	Type	Actors exposed	Exposure	Largest actor
RLUSD	Asset (loan token)	1	418.8M	Sentora RLUSD Main
`RLUSD / USD`	Oracle (feed)	1	418.8M	Sentora RLUSD Main
`BTC / USD`	Oracle (feed)	1	235.1M	Sentora RLUSD Main
kBTC	Asset (collateral)	1	235.1M	Sentora RLUSD Main
USDC	Asset (loan token)	10	132.6M	Vault Pendle Ecosystem USDC, 47.2M
`ETH / USD`	Oracle (feed)	1	106.9M	Sentora RLUSD Main
Ojo PT feed, behind PT-reUSD	Oracle (feed)	10	90.9M	—
PST-USDC exchange rate	Oracle (feed)	4	39.0M	—

The amounts count each position fully for every node it depends on, so they add up to more than the money at stake. They measure *how much depends on the node*, not a loss.

What it shows.

- **A shared feed under a whole vault.** One feed, `RLUSD / USD`, sits under all 418.8M Sentora lends.
- **A shared feed under several actors.** One Ojo feed sits under 90.9M of 10 different actors: the borrowing address, which can be liquidated, and nine lenders (vaults, and one lender the tool could not identify), which bear the bad debt.

A failure of either node hits every position above it **at the same time**, which no per-position analysis shows.

Step 2, with the engine: one shock end to end

Shock: the `RLUSD / USD` feed reads 10 % too high (then 20 %).

1. **Feed → oracles** (*reads*). The feed is the quote feed of all 7 market oracles. Morpho's standard oracle computes price = (base vault rate × base feeds) ÷ (quote vault rate × quote feeds), so every oracle price is divided by 1.1: a fall of 9.1 % (16.7 % at 20 %).
2. **Oracles → markets** (*priced by*). Each market's borrowers are revalued at the lower price: 48.8M of debt becomes liquidatable (55.5M at 20 %), and the positions the collateral no longer covers leave 0.26M of bad debt (3.82M at 20 %).
3. **Markets → vault** (*supplies*). Sentora supplies 100 % of each market, so it bears all of it: 0.06 % of its assets (0.83 % at 20 %).
4. **Vault → depositor** (*deposits in*). The largest depositor holds 32.6 %: a loss of 0.09M (1.25M at 20 %).

The borrowers of these markets are hit too, along the *borrows from* edge: 48.8M of debt is liquidated **although no collateral lost value**, because only the feed was wrong. That is the "oracle misreports" failure mode: real losses for borrowers (liquidation bonus) and for lenders (bad debt), with no real price move.

Steps 2 to 4 use the existing `market_risk.value()`, the same computation as point 4.3. What the engine adds is step 1, the routing from a feed to every oracle and market that reads it, and the ability to start from **any** node.

7. What could be implemented

In three steps, each useful on its own:

1. **Shared-node exposure** (S). Compute the table of section 6, step 1, from `graph.json`. Alert when one node carries more than a set share of a position, for example 50 % of a vault. This closes gap R3 and needs no engine.
2. **The propagation engine** (L):
   - **Data:** the graph as typed edges with their amounts (already in `graph.json`), plus each market's borrower positions (already in `market_risk`).
   - **Failure modes:** `price_drop(x)`, `depeg(x)`, `feed_misreport(±x)`, `freeze`, `total_loss`.
   - **Transmission functions:** one per edge type of section 4, starting with the four that carry most of the risk (*reads*, *priced by*, *supplies*, *deposits in*), each calling the existing function where one exists.
   - **Spreading:** DebtRank-style, each node once, ε pruning, USDC and Ethereum as terminal nodes.

- **Entry point:** `shock(node, mode, severity) → loss per actor, with the path that carried it`.
- **Acceptance test:** the engine reproduces the `RLUSD / USD` result of section 6, computed independently by `market_risk`.

3. **Use it everywhere** (M): as the computation behind `analyze --vault` (point 4.3), behind the what-if page (point 4.7), and to pick the scenarios worth listing (the nodes with the largest exposure in step 1).

8. Limits

- **Exposure is not loss.** Step 1 measures what depends on a node; the loss depends on the failure mode and severity.
- **Non-linear functions.** DebtRank was designed with linear weights; with liquidation thresholds the order in which shocks arrive and their size matter, so the engine must apply the edge functions, not a single weight.
- **Simultaneous shocks.** A real crisis moves several nodes at once (a depeg drains the pool and the oracle follows). The engine must accept a set of shocked nodes, not only one.
- **Coverage of the graph.** The engine is as good as the graph. Custom oracles without the standard getters are one opaque node, and the off-chain dependencies come from a hand-kept registry.
- **No feedback loop yet.** Liquidations sell collateral and push its price further; that loop needs the slippage curve of point 4.5.

Learn: what you need to understand

Graphs: nodes, edges, paths

A graph is a set of **nodes** (things) linked by **edges** (relations). In a **directed** graph each edge has a direction: here "A depends on B" is an edge from A to B. A **path** is a chain of edges; a node is **reachable** from another if a path joins them. Everything a node depends on is the set of nodes reachable from it. *In the project:* `graph.py`, `docs/graph-example.html`.

Exposure and look-through

Exposure is how much of a position depends on something. **Look-through** means following the chain to the end: a deposit in a vault is an exposure to the vault, but also, through it, to each market, each collateral and each price feed underneath. Look-through exposure is what makes shared dependencies visible.

Contagion and systemic risk

Contagion is a loss spreading from one participant to others through their links. **Systemic risk** is the risk that one failure, through contagion, damages much of the system. A node whose failure spreads widely is *systemically important*, the "too central to fail" of the DebtRank paper.

Centrality and blast radius

Centrality measures how important a node is to a network:

- **degree** counts its direct links;
- **betweenness** counts the shortest paths that cross it;
- **exposure-weighted** measures, like step 1 here, add the amounts at stake.

The **blast radius** of a node is everything its failure reaches. A feed with high betweenness and a large exposure is the first place to look.

Transmission functions and non-linearity

A transmission function turns a shock on one node into a loss on another. When it is **linear**, a 10 % shock gives half the loss of a 20 % shock. Liquidation makes it **non-linear**: no loss until a threshold, then a large one. Non-linear functions are why a small change in a shock or in a position's health can change the outcome completely. *In the project:* `propagate.loss`, `market_risk.value`, `docs/scenarios-and-breakeven.md` (the "cliff").

DebtRank

A measure of how much of a network's value a shock can destroy, built for banks lending to one another (Battiston et al., 2012). Each node's distress is its loss relative to its size; it spreads along weighted links, each node spreads once, and the result is the share of the network's value lost. Its two ideas used here are spreading each node once (to handle cycles) and stopping small paths (to keep the computation finite).

How a Morpho oracle combines feeds

Morpho's standard oracle (`MorphoChainlinkOracleV2`) returns the price of the collateral in the loan token as a product of up to two **base feeds** and an optional **base vault** rate, divided by up to two **quote feeds** and an optional **quote vault** rate. Two consequences:

- **A base-feed error** moves the price in the same direction.
- **A quote-feed error** moves it in the opposite direction: a feed reading too high lowers the price.

A quote feed shared by many oracles (`RLUSD / USD` under every Sentora market) moves all of them together.

Oracle failure modes

An oracle can fail in different ways, each with different victims:

- **It reads too low:** borrowers are liquidated without a real price move, and lenders may take bad debt.
- **It reads too high:** borrowers can borrow more than their collateral is worth, so lenders lose if someone exploits it.
- **It freezes or goes stale:** no liquidation happens while the real price moves, so bad debt builds silently.

A risk engine must treat these as separate shocks.

Check your understanding

1. **Why does a 10 % error in `RLUSD / USD` lower the oracle prices by 9.1 % and not 10 %?** It is a quote feed: prices are divided by 1.1, and 1 − 1/1.1 = 9.1 %.
2. **Why are 48.8M of debt liquidated in that scenario although no collateral lost value?** Liquidations follow the oracle price, and the oracle read too low because of the feed.
3. **Why can the exposure column of section 6 add up to more than the money at stake?** Each position is counted fully for every node it depends on: the same 235.1M depends on kBTC, on the `BTC / USD` feed and on RLUSD at once.

4. **Why must each node pass its shock on only once?** A cycle in the graph would otherwise amplify a shock without limit.
5. **Why not multiply percentages along a path?** Losses are non-linear: a borrower above its liquidation distance loses nothing, and one just beyond it loses most of its equity.

Further reading

- Stefano Battiston, Michelangelo Puliga, Rahul Kaushik, Paolo Tasca and Guido Caldarelli, "DebtRank: Too Central to Fail? Financial Networks, the FED and Systemic Risk", *Scientific Reports* 2, 541 (2012).
- Matthew Elliott, Benjamin Golub and Matthew O. Jackson, "Financial Networks and Contagion", *American Economic Review* 104(10), 2014.
- Mark Newman, *Networks* (Oxford University Press): graph concepts and centrality measures.
- Morpho's oracle documentation.
- In this repository: `ANALYSIS_MONITORING.md` §1 (the node model), `docs/graph-example.html`, `docs/vault-depositor-scenarios.md` (the same losses from the depositor's side).